Powered by mindgrasp

Summary

Java Foundations

Java, platform independence, and OOP

Java is a high-level, object-oriented language developed by Sun Microsystems in 1991 and released in 1995; Oracle acquired Sun in 2010. It is used for web, Android, enterprise, and game development. **Long-Term Support (LTS)** releases receive extended support from Oracle and other vendors.
Java's **Write Once, Run Anywhere (WORA)** model works by compiling source code into platform-independent **bytecode** in `.class` files. A platform-specific **Java Virtual Machine (JVM)** runs that bytecode on Windows, Linux, macOS, and other systems.
Java's core OOP concepts are:

- **Class:** Blueprint defining an object's properties and behavior.
- **Object:** An instance of a class, containing data and methods.
- **Encapsulation:** Bundling data and methods while controlling access to internal state.
- **Inheritance:** Reusing or extending another class's properties and behavior.
- **Polymorphism:** Allowing different objects to respond to the same method in different ways.
- **Abstraction:** Hiding implementation details and exposing essential functionality.

Java also emphasizes reliability through garbage collection, memory management, and runtime checks. **Just-In-Time (JIT) compilation** improves performance by compiling frequently executed bytecode into machine code.

First program and Java runtime components

A basic Java application defines a class and a `main` entry point:

- `public` makes `main` accessible to the JVM.
- `static` lets the JVM call it without creating an object.
- `void` means it returns no value.
- `String[] args` receives command-line arguments.
- `System.out.println(...)` uses the `System` class's `out` stream and `PrintStream.println()` to print a line.

Compile with `javac HelloWorld.java`; this creates `HelloWorld.class`. Run with `java HelloWorld`. The public class name and source filename must match. The JVM starts execution at `main`; `args` contains space-separated command-line inputs as strings, which programs can access by index and convert to other types. Check `args.length` before reading values to handle missing input.
The Java tools form a nested structure:

- **JDK (Java Development Kit):** Development toolkit containing the runtime, compiler (`javac`), JVM, standard libraries, and tools such as `javadoc`, `javap`, and `jdb`.
- **JRE (Java Runtime Environment):** Runtime libraries and JVM needed to run applications, without development tools.
- **JVM:** Executes bytecode, manages memory, and translates bytecode into machine-understandable instructions. It is created when `java` runs a program and ends when the program finishes.

Identifiers, data types, variables, and scope

An **identifier** names a class, method, variable, package, interface, or similar element. It may contain letters, digits, `$`, and `_`, but cannot begin with a digit or contain spaces or other punctuation. Identifiers are case-sensitive, cannot be reserved words, and have no formal maximum length. Descriptive names improve readability and maintenance; Java conventions

commonly use PascalCase for class names and camelCase for methods and variables.
Java is strongly typed. Its **primitive types** are `boolean`, `char`, `byte`, `short`, `int`, `long`, `float`, and `double`; their sizes and ranges are fixed except that boolean storage is JVM-specific. **Reference types** include strings, arrays, interfaces, enums, and user-defined classes. A reference holds access to an object rather than the object's value directly; an uninitialized reference field defaults to `null`, so it should be checked before use to avoid `NullPointerException`.
A variable has a type, name, value, and scope:

- **Local variables** are declared in methods, constructors, or blocks. They exist only within that scope and must be initialized before use.
- **Instance variables** belong to each object and receive default values if not explicitly initialized. They can be initialized in constructors or initialization blocks.
- **Static variables** belong to the class and have one shared copy, regardless of the number of objects. They can be initialized in static blocks and are commonly accessed through the class name.

Member variables are declared in a class but outside methods; local variables are temporary and disappear when their method or block ends. Curly braces generally define block scope.

Operators and interactive input

The assignment operator `=` assigns a right-hand value to a left-hand variable. Relational operators (`==`, `!=`, `<`, `<=`, `>`, `>=`) compare values and return booleans. Logical operators are `&&` (AND), `||` (OR), and `!` (NOT); `&&` and `||` short-circuit when the first condition determines the result.
For interactive console input, import `java.util.Scanner`, create a `Scanner` connected to `System.in`, and use methods such as `nextInt()`, `nextDouble()`, `next()`, or `nextLine()`. `hasNextInt()` can validate whether the next token is an integer before reading it.

Methods, Access Control, and Control Flow

Methods and access modifiers

A **method** is a reusable block of class-owned code. Its declaration includes an access modifier, return type, name, optional parameters, optional `throws` clause, and body. A method signature consists of its name and parameter types, number, and order—not its return type or exceptions. Methods can be built-in library methods or user-defined methods. An **instance method** operates on object-specific data and is called through an object. A **static method** belongs to the class and is called through the class name; it suits operations that do not depend on a particular object. Instance methods can be overridden and use runtime dispatch; static methods are resolved statically and can be hidden, not overridden. Methods improve reuse, modularity, abstraction, and maintainability.
Java access levels are:

- **private:** Accessible only within the declaring class.
- **default (package-private):** Accessible within the same package.
- **protected:** Accessible within the same package and by subclasses in other packages.
- **public:** Accessible from anywhere.

Private data can be exposed safely through public methods that validate or control access.

Decisions and loops

- **`if`:** Executes a block only when its condition is true.
- **`if-else`:** Selects between two paths.
- **`if-else-if`:** Tests conditions in order and executes the first matching branch; a final `else` handles no match.
- **`switch`:** Selects a branch based on a supported value, including integral types, `char`,

`String`, enums, and certain wrappers. `break` prevents execution from falling through into later cases; `default` handles unmatched values.

A **`for` loop** is useful when the iteration count is known: initialization runs once, the condition is checked before each iteration, and the update runs after each iteration. In **nested loops**, the inner loop completes for every outer-loop iteration. A **`while` loop** checks its condition before each iteration and may run zero times. A **`do-while` loop** checks afterward and therefore runs at least once.

`break` exits a loop or switch immediately. `continue` skips the remainder of the current loop iteration and proceeds to the next.

Arrays and Strings

Arrays

An **array** is a fixed-size object holding elements of one type, indexed from zero. Its length is available through `.length`; default element values depend on the type (for example, `0` for `int`). Arrays support fast index access and can be initialized with values directly or allocated with `new`. Accessing an invalid index throws `ArrayIndexOutOfBoundsException`. Arrays are useful for fixed collections, but cannot grow dynamically; collections such as `ArrayList` are alternatives when size changes.

Strings and mutable string classes

A **String** is an immutable sequence of characters: operations that appear to modify it produce another string. String literals are interned in the **String Constant Pool (SCP)**, allowing equal literals to share an object. `new String("...")` creates a distinct heap object; its literal may also be in the pool. Thus, two references can have equal content but different identities. Use `==` to compare object references and `.equals()` to compare string content. `intern()` returns the pooled reference for a string value.

Immutability supports safe sharing, thread safety, security, stable hash codes, and pool reuse. Common String methods include `length()`, `charAt()`, `equals()`, `equalsIgnoreCase()`, `indexOf()`, `lastIndexOf()`, `substring()`, `toUpperCase()`, `toLowerCase()`, `trim()`, `replace()`, `split()`, `contains()`, `startsWith()`, `endsWith()`, `toCharArray()`, and concatenation with `+` or `concat()`.

For repeated modifications:

- **StringBuffer** is mutable and synchronized, making it thread-safe but adding synchronization overhead. It supports operations such as `append()`, `insert()`, and `delete()`. Its default capacity is 16 characters; when exceeded, capacity grows according to `(current capacity × 2) + 2`.
- **StringBuilder** is mutable but not synchronized, so it is generally faster in single-threaded use. Use `StringBuffer` when shared concurrent access requires synchronization; use `StringBuilder` for ordinary repeated construction.

Object-Oriented Programming

Encapsulation and constructors

Encapsulation hides internal data—typically by making fields private—and provides controlled public operations such as getters, setters, deposits, or withdrawals. Validation inside those operations protects data integrity; for example, an account balance should not be directly editable.

A **constructor** initializes an object and runs automatically when `new` creates it. It has the class's name and no return type. If no constructor is declared, the compiler supplies a no-argument constructor. A constructor's first statement may call `this(...)` for another constructor in the same class or `super(...)` for a parent constructor; if omitted, a no-

argument `super()` is inserted when applicable. Constructor chaining avoids duplicated initialization. Constructors can use access modifiers but cannot be `static`, `final`, `abstract`, or `synchronized`.

Inheritance, abstraction, and polymorphism

Inheritance, declared with `extends`, lets a subclass reuse and extend a superclass. Java supports single, multilevel, and hierarchical class inheritance. It does not allow a class to extend multiple classes because conflicting inherited implementations create ambiguity; a class can implement multiple interfaces instead. If interface default methods conflict, the implementing class must resolve the conflict explicitly. A subclass can use inherited accessible members, add fields or methods, override instance methods, hide static methods, and call parent constructors. Private parent members are not directly accessible to subclasses.
An **abstract class** cannot be instantiated. It can combine abstract methods (without bodies) with concrete methods, fields, constructors, static methods, and final methods. Subclasses must implement inherited abstract methods unless they too are abstract. **Abstraction** exposes what an operation does while hiding how it works; abstract classes provide shared structure while allowing specialized implementations.
Polymorphism allows a common reference type to represent different concrete objects. The compiler checks that the method exists on the reference type; at runtime, an overridden instance method is selected according to the actual object (**dynamic method dispatch**).

- **Overloading (compile-time polymorphism):** Same method name, different parameter number, types, or order. Return type alone cannot distinguish overloads.
- **Overriding (runtime polymorphism):** A subclass supplies a method with the same signature as an inherited instance method. The overriding method cannot reduce access; `final` methods cannot be overridden. A covariant return type may be a subtype of the parent method's return type. Static methods are hidden, not overridden; constructors are not overridden.

Interfaces, Exceptions, and Database Access

Interfaces

An **interface** defines a contract that implementing classes must fulfill. Traditionally, its fields are `public static final` constants and its methods are `public abstract`. A class uses `implements`; interfaces can extend multiple interfaces. Interfaces cannot be instantiated or have constructors, but references of interface type can point to implementing objects.
Java 8 added **default** methods with implementations and **static** methods; Java 9 added **private** helper methods for reuse inside interface methods. A **functional interface** has exactly one abstract method (plus any number of default or static methods), enabling lambdas and method references. `@FunctionalInterface` asks the compiler to enforce this rule. Examples include `Runnable`, `Callable`, `Comparable`, and `ActionListener`. A **marker interface** has no methods and tags a class for special handling, as with `Serializable` and `Cloneable`. Nested interfaces are declared inside another type.

Exception handling

An **exception** disrupts normal execution; examples include invalid array indexes, division by zero, missing files, and null access. Unhandled exceptions stop normal execution. All throwable types derive from `Throwable`: **Exception** represents conditions applications may handle, while **Error** represents serious runtime problems such as stack overflow or memory exhaustion.

- **Checked exceptions** must be handled or declared, such as `IOException`, `SQLException`, and `FileNotFoundException`.

- **Unchecked exceptions** are runtime exceptions not checked by the compiler, such as `ArithmeticException`, `NullPointerException`, `ClassCastException`, and `ArrayIndexOutOfBoundsException`.
- **Custom exceptions** are created by extending `Exception` or `RuntimeException`, adding a message constructor, and throwing and handling the exception as appropriate.

A `try` block contains risky code; `catch` handles matching exceptions; `finally` runs cleanup code after the try/catch flow. Multiple catch blocks can handle different exception types. The keywords are `try`, `catch`, `throw` (explicitly raise an exception), `throws` (declare possible exceptions), and `finally`.

JDBC

JDBC (Java Database Connectivity) is a standard API for connecting Java applications to relational databases, executing SQL, and processing results. Its main elements include `DriverManager`, `Connection`, `Statement`, `PreparedStatement`, `CallableStatement`, `ResultSet`, and `SQLException`. JDBC supports two-tier access (application directly to database) and three-tier access (through a middle server).
Driver categories include bridge, native API, network protocol, and Type-4 thin drivers; Type 4 is commonly used in modern applications. A typical connection workflow is:
1. Load the driver when required.
1. Obtain a `Connection` through `DriverManager`.
1. Create a statement.
1. Execute SQL (`executeUpdate()` for data changes; query methods return results).
1. Close statements and connections.

`PreparedStatement` supports parameterized queries and is preferable for dynamic input, including reducing SQL-injection risk. JDBC applications should handle `SQLException` and release resources reliably.

Collections and Streams

Collection Framework

The **Collection Framework**, introduced in JDK 1.2, provides shared interfaces and implementations for groups of objects. `Iterable` supplies iteration; `Collection` is the root interface for collections. **`Collection` is an interface; `Collections` is a utility class** with static operations such as synchronized and unmodifiable wrappers. **`Map`** is a separate key-value abstraction.

- **List:** Ordered and allows duplicates. `ArrayList` uses a resizable array, offers O(1) indexed access, and shifts elements for middle insertions/removals. `LinkedList` uses doubly linked nodes; access by index is O(n), while insertion/removal at an already-reached position is O(1). Traversal to a middle position still costs O(n).
- **Set:** Rejects duplicates. `HashSet` is unordered and hash-based; `LinkedHashSet` preserves insertion order; `TreeSet` sorts by natural ordering or a comparator and generally performs operations in O(log n). `TreeSet` uses comparison to identify duplicates and does not accept null elements.
- **Map:** Keys are unique; values may repeat. `HashMap` is unordered, `LinkedHashMap` preserves insertion order, and `TreeMap` sorts by key. `ConcurrentHashMap` supports concurrent access without locking the entire map.

Hash-based collections use `hashCode()` to locate a bucket and `equals()` to check equality. Equal objects must have equal hash codes. `HashSet` is backed by a `HashMap`; `HashMap` handles collisions by checking keys with `equals()`, using linked structures or tree bins for heavily populated buckets in modern Java. Its default capacity is 16 and load factor 0.75; exceeding the threshold triggers resizing and rehashing.
Collections can be traversed with enhanced `for` loops or `Iterator`. Use a collection's

supported iterator operations when modifying during traversal to avoid concurrent-modification problems. Choose implementations according to ordering, uniqueness, access patterns, and concurrency needs.

Stream API

The Java 8 **Stream API** processes object collections declaratively; it is distinct from byte-oriented `InputStream` and `OutputStream`. A stream does not store data. It builds a pipeline of operations, usually using lambdas, and may run sequentially or in parallel.
Intermediate operations return streams and are lazy: `filter`, `map`, `flatMap`, `distinct`, `limit`, `peek`, `sorted`, and `skip`. **Terminal operations** trigger processing and consume the stream: `forEach`, `toArray`, `reduce`, `collect`, `min`, `max`, and `count`. A stream cannot be reused after a terminal operation. Lazy pipelines can process elements in a single pass and short-circuit when a result is determined.
Collectors include `toList`, `toSet`, `toCollection`, `joining`, `counting`, `groupingBy`, and `partitioningBy`. Primitive mappings such as `mapToInt`, `mapToLong`, and `mapToDouble` enable numeric operations. Parallel streams can help with suitable large workloads, but require attention to overhead, ordering, and thread safety.

Build Tools, Testing, and Concurrency

Maven and JUnit

Maven automates builds and dependency management through `pom.xml`, which defines project coordinates, packaging, dependencies, repositories, and plugins. It checks the local repository first, then configured remote repositories and Maven Central. Common commands are `mvn clean`, `compile`, `test`, `package`, `install`, and `deploy`. Standard Maven projects use conventional source and test directories.
JUnit supports unit testing—testing small units such as methods or classes—within the testing phase of the Software Development Life Cycle. JUnit 4 uses annotations such as `@Test`, `@Before`, and `@After`; tests use assertions to verify behavior and can be run through an IDE or build tool. Automated tests catch defects early and support regression testing. **Code coverage** measures executed code, but meaningful tests matter more than achieving an arbitrary percentage; 80% is a common target, while 100% may be impractical.

Threads and concurrency

A **thread** is a unit of execution within a process; the JVM starts the `main` thread. Threads can be created by extending `Thread` or implementing `Runnable`; `Runnable` is more flexible, while `ExecutorService` is generally preferred for managed thread pools. Calling `start()` schedules a new thread; calling `run()` directly is an ordinary method call. A completed thread cannot be restarted.
Thread states include new, runnable, running, blocked, waiting/timed waiting, and terminated. A waiting thread must pass through runnable before running again. `join()` makes the calling thread wait for another thread to finish.
Synchronization protects shared mutable state from race conditions. `synchronized` methods or blocks use an object or class monitor; a thread must acquire the lock before entering. `ReentrantLock` offers explicit locking, timed attempts, and `tryLock()`. `ReadWriteLock` permits concurrent readers but exclusive writing; `Semaphore` limits simultaneous access using permits. `wait()`, `notify()`, and `notifyAll()` are methods of `Object`; `wait()` releases the monitor, whereas `sleep()` pauses without releasing it. These coordination methods require the relevant monitor. Producer-consumer systems can also use blocking queues.
The **Executor Framework** separates task submission from thread management. `ExecutorService` submits work and manages shutdown; `Callable<V>` returns a result and

can throw checked exceptions, while `Runnable` does not return a result. A `Future<V>` represents an asynchronous result and supports checking, cancellation, and retrieval. Common pools include fixed, cached, single-thread, and scheduled pools. Thread pools avoid the cost of creating a new thread for every task.

Spring Framework and Spring Boot

Spring Core and dependency injection

Spring is a modular framework for Java applications. Its modules include Core, Beans, Context, Web/MVC, JDBC, ORM, Data JPA, Security, Test, Boot, Cloud, Batch, and Scheduling. **Inversion of Control (IoC)** transfers object creation and dependency management to the Spring container. A **Spring bean** is an object managed by that container.
The main containers are **BeanFactory**, a lightweight container with lazy bean creation, and **ApplicationContext**, a richer container commonly used in enterprise applications, supporting broader integration and XML, annotation, or Java configuration. Beans can be registered through component annotations such as `@Component`, `@Service`, and `@Repository`, or through `@Bean` methods in configuration classes.
Bean lifecycle stages include instantiation, dependency injection, initialization, use, and destruction. Initialization and cleanup can use annotations such as `@PostConstruct` and `@PreDestroy`, lifecycle interfaces, or configured methods. Scopes include singleton (default, one per container), prototype (new instance per request from the container), request, session, and application.
Dependency Injection (DI) supplies a bean's dependencies:

- Constructor injection suits required dependencies and supports immutability and testing.
- Setter injection suits optional or reconfigurable dependencies.
- Field injection is concise but makes testing and encapsulation harder, so constructor injection is generally preferred.

`@Qualifier` selects a particular bean when multiple implementations match. **Profiles** activate environment-specific beans or configuration, such as separate development and production data sources. Externalized values can be injected from properties, while production secrets should be stored securely.

Spring Data and persistence

Spring Data reduces persistence boilerplate through repositories, CRUD operations, query derivation, paging, sorting, auditing, and support for relational and NoSQL stores. A JPA **entity** maps a Java class to a database table; `@Id` marks its primary key and `@GeneratedValue` selects a generation strategy (`IDENTITY`, `SEQUENCE`, `TABLE`, or `AUTO`). `JpaRepository` provides CRUD, paging, and sorting capabilities.
Simple queries can be derived from method names; `@Query` supports custom JPQL or native SQL. `@Transactional` defines a transaction boundary, typically at the service layer so multi-repository business operations succeed or fail atomically.
JPA specifies persistence rules; **Hibernate** is an implementation of JPA; **Spring Data JPA** provides a higher-level repository abstraction. Common annotations include `@Entity`, `@Table`, `@Column`, `@JoinColumn`, relationship annotations, `@Transient`, and `@Version`. Version-based **optimistic locking** detects conflicting updates at write time and raises an optimistic-lock exception rather than silently overwriting another change.

Spring MVC and web requests

Spring MVC separates model (data/business logic), view (presentation), and controller (request handling). The **DispatcherServlet** is the front controller: it receives requests, uses `HandlerMapping` to select a controller, delegates execution through a handler adapter, resolves a view through a `ViewResolver`, and returns the rendered response.

`@Controller` commonly returns views; `@RestController` combines `@Controller` and `@ResponseBody` for data responses. `@RequestMapping` and shortcuts such as `@GetMapping` and `@PostMapping` map URLs and HTTP methods. `@RequestParam` binds query parameters; `@PathVariable` binds URL path segments; `@RequestBody` binds request content to an object. Controllers may return a view name, `ModelAndView`, or `ResponseEntity` for control over REST response body, status, and headers. `@ExceptionHandler` handles exceptions locally; `@ControllerAdvice` centralizes handling across controllers. HTTP methods include GET (retrieve), POST (create), PUT (replace), PATCH (partially update), DELETE, HEAD, and OPTIONS. Status codes are grouped as 2xx success, 3xx redirection, 4xx client errors, and 5xx server errors.

Supporting modules: security, AOP, validation, and caching

Spring Security distinguishes authentication (who the user is) from authorization (what the user may do). Servlet filters intercept requests before controllers; Spring Security uses a filter chain. In session authentication, successful login stores authentication in a security context associated with the session. In **JWT** authentication, the client sends a bearer token on each request and the server validates it without a session. **OAuth 2.0** delegates authentication or authorization to an identity provider such as Google.
Method security can use `@PreAuthorize`, `@PostAuthorize`, `@Secured`, and `@RolesAllowed`; `@PreAuthorize` supports fine-grained expressions. Passwords should use a secure encoder such as BCrypt rather than plain-text comparison.
AOP (Aspect-Oriented Programming) modularizes cross-cutting concerns such as logging, security, auditing, and transactions. An `@Aspect` applies advice at matched join points; annotations include `@Before`, `@After`, `@AfterReturning`, `@Around`, and reusable `@Pointcut`.
Validation checks object data before processing. `@Valid` triggers annotation-based validation; `BindingResult` holds errors and must immediately follow the validated parameter. A custom validator can implement rules beyond standard annotations.
Caching stores frequently requested, slowly changing data to reduce latency and backend load. Strategies include cache-aside, write-through, write-behind, and read-through. Cache hits return stored data; misses fetch and may populate it. TTL, LRU, and LFU govern expiration or eviction. Spring's `@EnableCaching` activates caching; `@Cacheable` stores method results, `@CachePut` updates cache entries while executing the method, and `@CacheEvict` removes entries.

Spring Boot

Spring Boot builds on Spring to accelerate setup and deployment through auto-configuration, starter dependencies, embedded servers, sensible defaults, and production monitoring. `@SpringBootApplication` combines configuration, auto-configuration, and component scanning. Starters bundle common dependencies, such as web, JDBC, JPA, security, and testing libraries. Spring Boot normally uses embedded Tomcat, which can be replaced with another server.
Configuration is externalized in `application.properties` or `application.yml`; profiles such as `dev`, `test`, and `prod` can override environment-specific settings. Spring Initializr generates project scaffolding. **Actuator** provides operational endpoints such as health, metrics, information, and environment details. Testing annotations include `@SpringBootTest`, `@WebMvcTest`, and `@DataJpaTest`.

Microservices and Kafka

Microservices architecture

A **monolith** packages presentation, business logic, and data access as one deployable unit. It

is simple for small applications but can become tightly coupled, difficult to maintain, technology-constrained, and costly to scale or redeploy. **Microservices** organize an application as small, independently deployable services around business capabilities or bounded contexts. They can scale and evolve independently and use different technologies, but introduce distributed-system complexity, duplicated configuration, and reduced whole-system visibility.
Supporting components are optional and chosen by need: API gateways route and filter requests; registries enable service discovery; configuration servers centralize settings; Feign simplifies service calls; Kafka or RabbitMQ supports asynchronous messaging; Redis provides caching; circuit breakers limit repeated failing calls; load balancers distribute traffic; tracing, monitoring, dashboards, and centralized logging support operations.
Migration can proceed incrementally: assess the monolith, identify business capabilities, establish infrastructure, extract services using the **Strangler** approach, build resilient communication, separate service data, refactor security, test, monitor, and retire the monolith when migration is complete.

Apache Kafka

Kafka is a durable, high-throughput publish-subscribe messaging platform. It stores messages on disk for later consumption and supports multiple producers and consumers. Its performance relies on batching, sequential disk access, zero-copy transfer, and horizontal scaling through partitions.
A **broker** stores and serves records. A **topic** is a named log divided into **partitions**, which enable parallelism and distribution across servers. Records with the same key are routed to the same partition, preserving their order within that partition. Producers publish records; consumers subscribe and track progress with **offsets**. In a consumer group, each partition is assigned to at most one consumer at a time, allowing parallel processing across partitions; extra consumers may remain idle.
A producer serializes a record, selects a partition (explicitly or by key), batches it, and sends it to a broker. A successful acknowledgement can include topic, partition, and offset metadata; failures may be retried. Durability and delivery behavior depend on replication, acknowledgements, and producer settings. Kafka's advanced operational concerns include ISR replicas, consumer-group rebalancing, committed offsets, idempotence and transactions, schema compatibility, and monitoring of lag, throughput, disk, and memory.

Microservices Design Patterns

Service data and consistency patterns

Database per Microservice gives each service ownership of its persistent data, accessed through APIs rather than direct cross-service database access. This supports independent deployment, scaling, failure isolation, and **polyglot persistence**—for example, document storage for product catalogs, a key-value store for carts, and a relational database for orders.
CQRS (Command Query Responsibility Segregation) separates commands that change data from queries that read it. Separate write and read stores can be optimized for their workloads, especially in read-heavy systems. Events can update the read store asynchronously, but this introduces **eventual consistency**: replicas may temporarily differ.
Event Sourcing stores the sequence of events that produced a state rather than only the latest state. The event store becomes the source of truth; events can be replayed to rebuild state or create materialized read views. This supports history and recovery, but requires careful event handling and projections.

Distributed workflows and client access

A **Saga** coordinates a distributed transaction as a sequence of local transactions. Each

successful step triggers the next; failures invoke compensating transactions to undo prior effects. **Choreography** uses services' published events to trigger one another and suits simpler workflows. **Orchestration** uses a coordinator to direct steps and track outcomes, making complex workflows easier to manage.
Backend for Frontend (BFF) adds a client-specific layer that calls multiple services and aggregates or transforms their responses for a particular interface, such as web or mobile. It reduces frontend complexity but should be used where client needs justify the extra layer.
An **API Gateway** is a client-facing entry point that routes requests and may handle authentication, SSL termination, caching, and other cross-cutting tasks. It abstracts internal services, but a single overloaded gateway can become a bottleneck or single point of failure; deployment and responsibilities should reflect the system's needs.

Migration, resilience, configuration, and contracts

The **Strangler Pattern** gradually replaces monolith functionality: identify a business capability, implement it as a service, route some traffic to it, increase traffic as it matures, monitor results, and eventually retire the old implementation.
A **Circuit Breaker** monitors calls to a failing service. After failures cross a threshold, it stops further calls temporarily and can return an error, fallback, or cached response, limiting cascading failures.
Externalized Configuration stores settings outside service deployments—in a configuration server, database, filesystem, or environment variables—so many service instances can share updated configuration without rebuilding each one.
Consumer-Driven Contract Testing has consumers define expected provider inputs and outputs in a shared contract. Providers implement and test that contract, publish it, and consumers verify compatibility. Agreeing on expectations early reduces integration failures.

Key Takeaways

- Java compiles source into bytecode; the JVM enables platform-specific execution.
- Understand the distinctions among primitive and reference types, local and member variables, static and instance members, and `==` versus `.equals()`.
- OOP relies on encapsulation, inheritance, abstraction, and polymorphism; interfaces support contracts and multiple implementation types.
- Exceptions, JDBC, collections, streams, Maven, JUnit, and concurrency provide core tools for building and testing Java applications.
- Spring IoC manages beans and dependencies; Spring Boot reduces configuration, while Spring MVC, Data, Security, AOP, validation, and caching address common application concerns.
- Microservices improve independent deployment and scaling but require deliberate handling of communication, data ownership, resilience, observability, and distributed transactions.
- Kafka provides durable partitioned messaging; microservices patterns such as CQRS, Event Sourcing, Saga, BFF, API Gateway, Circuit Breaker, and Strangler address recurring architectural problems.